

The Alcove & Adder Programming Manual

Alcove Team

The Alcove & Adder Reference Manual: A Technical Deep Dive

Welcome to the definitive programmer’s reference and architecture manual for the **Alcove** and **Adder** languages. This book is generated dynamically: every single code block is executed by the actual compiled `alcove` and `adder` binaries, verifying both compiler correctness and execution output in real-time.

Chapter 1: VM Architecture & Memory Design

1.1 One VM, Two Symmetrical Dialects

Alcove and Adder are two surface syntaxes compiling to the same register-based bytecode Virtual Machine: * **Alcove** represents the classic Lisp-style S-expression syntax. * **Adder** is a Python-inspired block-based syntax that compiles down to the exact same S-expressions and runs on the same VM.

Because they compile to the same underlying S-expressions, they share the same global namespace, the same FFI bindings, the same garbage collection behaviors, and can transparently import (**require**) and call each other.

1.2 Pointer Tagging Layout

On 64-bit platforms, allocations returned by `malloc` are aligned to 8 bytes, which leaves the lowest 3 bits of a pointer address as 000. Alcove uses this alignment to store immediate values (fixnums and characters) directly inside the pointer variable (`exp_t *`) itself, skipping the heap: `* 000 (TAG_PTR)`: Standard heap-allocated node (e.g. lists, dicts, vectors). `* 001 (TAG_FIX)`: 61-bit signed immediate integer. `* 010 (TAG_CHAR)`: Unicode character immediate.

(TAG_PTR)	Pointer Address (61 bits)	0	0	0
(TAG_FIX)	Signed Integer Value (61 bits)	0	0	1

(TAG_CHAR)	Unicode Character Code (61 bits)	0	1	0
------------	----------------------------------	---	---	---

This immediate value tagging avoids memory allocation, refcount tracking, and collection overhead.

1.3 Memory Management & Teardown

Alcove uses a fast-path reference counting model. To handle cycles formed by local closures capturing their parent frames, the VM implements a cycle-breaking pass (`env_break_self_cycle`) on environment destruction. For user-defined structures, (`heap-stats`) is provided to track live allocation cells.

Chapter 2: Language Dialect Syntax Comparisons

2.1 Symmetrical Comparison

Let's see how identical expressions are written in Lisp (Alcove) vs Python-style (Adder).

Alcove (Lisp) In [1]:

```
(+ 1 2)
```

Out [1]:

```
3
```

In [2]:

```
(is 1 1)
```

Out [2]:

```
t
```

Adder In [1]:

```
1 + 2
```

Out [1]:

```
3
```

In [2]:

```
1 is 1
```

Out [2]:

```
t
```

2.2 Infix Notation Rewriting Mechanics

Unlike traditional Lisp, Alcove features infix rewriting. * **Compile-time:** An expression `(A op B)` where `op` is a known operator is compiled as `(op A B)` if `A` is statically non-callable (e.g., a literal, symbol, or type-hinted parameter). * **Runtime:** If the compiler cannot determine callability statically, it compiles a generic call. At runtime, the evaluator and VM dispatch `(A op B)` as `(op A B)` if `A` evaluates to a non-callable value.

Chapter 3: Deep Dive into Core Data Types

3.1 Numbers Tower

Alcove supports 61-bit fixnums, IEEE-754 double floats, exact fractional Ratios, and fixed-precision Decimals.

Alcove (Lisp) In [1]:

```
(/ 1 3)
```

Out [1]:

```
0
```

In [2]:

```
(rational? 1/3)
```

Out [2]:

```
t
```

In [3]:

```
(decimal "1.25")
```

Out [3]:

```
1.25m
```

In [4]:

```
(decimal? (decimal "1.25"))
```

Out [4]:

```
t
```

Adder In [1]:

```
1/3
```

Out [1]:

1/3

In [2]:

```
rational?(1/3)
```

Out [2]:

t

In [3]:

```
decimal("1.25")
```

Out [3]:

1.25m

In [4]:

```
decimal?(decimal("1.25"))
```

Out [4]:

t

3.2 Native Collections

Alcove provides multiple native collections: * **Lists**: Linked cons cells. * **Vectors**: Flat arrays with double-ended queue operations. * **Hash Maps**: Mutable hash tables. * **Persistent Maps**: Immutable Hash Array Mapped Tries (**hamt**). * **Blobs**: Byte arrays.

Alcove (Lisp) In [1]:

```
(list 1 2 3)
```

Out [1]:

(1 2 3)

In [2]:

```
(vec 10 20 30)
```

Out [2]:

#[20 20 20 20 20 20 20 20 20 20]

In [3]:

```
(hash-map "name" "alcove" "version" 2)
```

Out [3]:

{"version" 2, "name" "alcove"}

In [4]:

```
(hamt "a" 1 "b" 2)
```

Out [4]:

```
{"b" 2, "a" 1}
```

In [5]:

```
(make-blob "hello")
```

Out [5]:

```
#b"hello"
```

Adder In [1]:

```
list(1 2 3)
```

Out [1]:

```
(1 2 3)
```

In [2]:

```
vec(10 20 30)
```

Out [2]:

```
#[20 20 20 20 20 20 20 20 20 20]
```

In [3]:

```
hash-map("name" "adder" "version" 2)
```

Out [3]:

```
{"version" 2, "name" "adder"}
```

In [4]:

```
hamt("a" 1 "b" 2)
```

Out [4]:

```
{"b" 2, "a" 1}
```

In [5]:

```
make-blob("hello")
```

Out [5]:

```
#b"hello"
```

3.3 Stateful Generators

Generators provide stateful yield-style lazy sequence iteration.

Alcove (Lisp) In [1]:

```
(setf g (gen-range 1 5))
```

Out [1]:

```
{"gs" 1, "gk" 1, "gc" 1, "ge" 5}
```

In [2]:

```
(gen-next! g)
```

Out [2]:

```
1
```

In [3]:

```
(gen-next! g)
```

Out [3]:

```
2
```

Adder In [1]:

```
g = gen-range(1 5)
```

Out [1]:

```
{"gs" 1, "gk" 1, "gc" 1, "ge" 5}
```

In [2]:

```
gen-next!(g)
```

Out [2]:

```
1
```

In [3]:

```
gen-next!(g)
```

Out [3]:

```
2
```

Chapter 4: Advanced Scopes, Control Flows, & Recursion

4.1 Local Bindings & Assignment

We can bind variables locally using `let` or `=` assignment.

Alcove (Lisp) In [1]:

```
(let (x 10 y 20) (+ x y))
```

Out [1]:

30

Adder In [1]:

```
let (x 10 y 20):  
  x + y
```

Out [1]:

30

4.2 Conditionals & Pattern Matching

We can use `if`, `cond`, `case`, and structural pattern matching `match`.

Alcove (Lisp) In [1]:

```
(cond (is 1 2) "no" (is 1 1) "yes")
```

Out [1]:

"yes"

In [2]:

```
(match (list 1 2) (list x y) (+ x y) "fallback")
```

Out [2]:

3

Adder In [1]:

```
cond:  
  1 is 2  
  "no"  
  1 is 1  
  "yes"
```

Out [1]:

"yes"

In [2]:

```
match list(1 2):  
  list(x y)  
  x + y
```

```
-  
"fallback"
```

Out [2]:

3

4.3 Loops & Recursion

Supported loop constructs include `while`, `repeat`, `for`, and `each`.

Alcove (Lisp) In [1]:

```
(let (i 0)  
  (while (< i 3)  
    (prn i)  
    (= i (+ i 1))))
```

Stdout:

0
1
2

Out [1]:

nil

Adder In [1]:

```
let i 0:  
  while (i < 3):  
    prn(i)  
    i = i + 1
```

Stdout:

0
1
2

Out [1]:

nil

4.4 Escape Continuations (`call/cc` & `defc`)

Escape continuations allow capturing the current control flow, enabling early returns, custom aborts, and exception handling.

Alcove (Lisp) In [1]:

```
(defc search (val lst)
  (each x lst
    (if (is x val) (return t)))
  nil)
```

Out [1]:

```
#<procedure:search@...>
```

In [2]:

```
(search 42 (list 10 42 100))
```

Out [2]:

```
t
```

Adder In [1]:

```
defc search (val lst):
  each x lst:
    if (x is val):
      return(t)
  nil
```

Out [1]:

```
#<procedure:search@...>
```

In [2]:

```
search(42 list(10 42 100))
```

Out [2]:

```
t
```

Chapter 5: Metaprogramming (Macros)

Macros are defined using `defmacro`. They manipulate S-expressions and expand at compile time.

Alcove (Lisp) In [1]:

```
(defmacro when (cond body)
  (list 'if cond body nil))
```

Out [1]:

```
#<macro:when@...>
```

In [2]:
`(macroexpand-1 '(when t (prn "hello")))`

Out [2]:
`(when t (prn "hello"))`

Adder In [1]:
`defmacro when (cond body):
 list('if cond body nil)`

Out [1]:
`#<macro:when@...>`

In [2]:
`macroexpand-1('(when t prn("hello")))`

Out [2]:
`(when t (prn "hello"))`

Chapter 6: Polymorphism (Structs & Multimethods)

Records are defined via `defstruct`. Custom type polymorphism is supported via `defmulti` and `defmethod`.

Alcove (Lisp) In [1]:
`(defstruct point x y)`

Out [1]:
`point`

In [2]:
`(= p (point 3 4))`

Out [2]:
`{"__type__" point, "x" 3, "y" 4}`

In [3]:
`(point-x p)`

Out [3]:
`3`

In [4]:

```

(point? p)
Out [4]:
t
In [5]:
(defmulti greet type-of)
Out [5]:
greet
In [6]:
(defmethod greet 'point (p) "hello point!")
Out [6]:
{"point" #<procedure@...>}
In [7]:
(defmethod greet 'int (n) "hello int!")
Out [7]:
{"int" #<procedure@...>, "point" #<procedure@...>}
In [8]:
(greet p)
Out [8]:
"hello point!"
In [9]:
(greet 42)
Out [9]:
"hello int!"

Adder In [1]:
defstruct point x y
Out [1]:
point
In [2]:
p = point(3 4)
Out [2]:

```

```

{"__type__" point, "x" 3, "y" 4}
In [3]:
point-x p
Out [3]:
3
In [4]:
point? p
Out [4]:
t
In [5]:
defmulti greet type-of
Out [5]:
greet
In [6]:
defmethod greet 'point (p):
  "hello point!"
Out [6]:
{"point" #<procedure@...>}
In [7]:
defmethod greet 'int (n):
  "hello int!"
Out [7]:
{"int" #<procedure@...>, "point" #<procedure@...>}
In [8]:
greet(p)
Out [8]:
"hello point!"
In [9]:
greet(42)
Out [9]:
"hello int!"

```

Chapter 7: High-Performance JIT Compiler

The VM contains an integrated JIT compiler that generates native instructions (AMD64/ARM64) at runtime. It accelerates recursion patterns like Ackerman loops, simple step loops, and Fibonacci layouts: * **W^X Compliance**: Pages are mapped Write-only during compilation, then switched to Read/Execute before invocation. * **Fast Call paths**: Skips frame creation overhead for tight numeric loops.

Chapter 8: Systems Integration & FFI

8.1 Foreign Function Interface (FFI)

Alcove supports loading shared libraries (.so / .dylib) dynamically, mapping C structures, and exporting Lisp procedures as C callback function pointers.

8.2 Safe Mode Sandbox

Invoking `./alcove --safe` enables a restrictive sandbox: * Blocks all raw filesystem and network calls. * Bypasses to unsafe builtins (like FFI or shell command gates) throw an immediate security exception.